

Desenvolvimento de ERP para Cooperativas

https://gemini.google.com/app/d35d55e0611844d7

User prompt: bom dia, conseguimos por aqui criar um app de gestão para cooperativas que seja 100% funcional, sistema de ERP?

Response: Bom dia! A resposta curta é **não diretamente por este chat**, mas conseguimos fazer quase todo o trabalho de projeto, arquitetura e inteligência por trás dele.

Um ERP (Enterprise Resource Planning) 100% funcional exige um ambiente de execução dedicado com banco de dados relacional, servidores, autenticação segura e interfaces de usuário (front-end). Como um modelo de linguagem, eu não consigo hospedar, rodar ou implantar uma aplicação dessa magnitude sozinho.

No entanto, posso atuar como seu **arquiteto de software e desenvolvedor principal** durante todo o processo.

Como podemos construir isso juntos:

- **Modelagem de Banco de Dados:** Criação de schemas SQL completos para gerenciar cooperados, cotas-partes, assembleias, fluxo financeiro, estoque e distribuição de sobras/perdas.
- **Geração de Código:** Escrita de código para o back-end (Node.js, Python, Java, C#) e front-end (React, Vue, Flutter) da aplicação.
- **Regras de Negócio e Compliance:** Definição da lógica fiscal, contábil e regulatória específica para o setor cooperativista (ex: Lei das Cooperativas no Brasil - Lei 5.764/71).
- **Documentação e APIs:** Especificação de APIs RESTful/GraphQL e documentação técnica do sistema.

Arquitetura recomendada para o seu ERP:

Camada	Tecnologias Sugeridas	Função no ERP
Front-end	React, Vue.js ou Flutter	Interface para gestores e cooperados (Web/Mobile)
Back-end	Node.js (NestJS) ou Python (FastAPI/Django)	Regras de negócio, relatórios e processos do ERP
Banco de Dados	PostgreSQL	Armazenamento relacional e seguro de dados financeiros e cadastrais
Infraestrutura	AWS, Google Cloud ou Azure	Hospedagem, rotinas de backup e segurança

Para darmos o primeiro passo, qual é o ramo da cooperativa (ex: agropecuária, crédito, saúde, trabalho) e qual linguagem ou tecnologia você pretende usar no desenvolvimento?

User prompt: Somos uma cooperativa de trabalho de materiais recicláveis

Response: Excelente nicho. O ERP de uma cooperativa de reciclagem precisa focar em rastreabilidade de materiais, controle de triagem, pesagem na balança, produtividade por cooperado e divisão justa das sobras.

Módulos Essenciais do ERP de Reciclagem

- **Entrada e Pesagem:** Integração com balança rodoviária/plataforma, registro de doadores/geradores de resíduos e emissão de ticket de pesagem.
- **Triagem e Estoque:** Controle do material bruto vs. material processado (prensado, fardado, moído) e controle de perdas/rejeito.
- **Produtividade dos Cooperados:** Registro de horas trabalhadas ou quilogramas triados por cooperado para o cálculo das diárias/comissões.
- **Venda e Comercialização:** Gestão de compradores (indústrias, recicladores), contratos de fornecimento e emissão de notas fiscais.
- **Financeiro e Rateio:** Controle de caixa, pagamento de despesas operacionais e cálculo automático do rateio de sobras líquido com base na produção individual ou horas trabalhadas.

Estrutura de Dados Básica (Banco de Dados)

Para começar a desenhar o sistema, precisaremos de tabelas fundamentais no PostgreSQL:

```
-- Cadastro de Cooperados
CREATE TABLE cooperados (
  id SERIAL PRIMARY KEY,
  nome VARCHAR(100) NOT NULL,
  cpf VARCHAR(14) UNIQUE NOT NULL,
  data_admissao DATE NOT NULL,
  ativo BOOLEAN DEFAULT TRUE
);

-- Entrada de Materiais (Recebimento)
CREATE TABLE entradas_material (
  id SERIAL PRIMARY KEY,
  fornecedor_origem VARCHAR(100),
  peso_bruto NUMERIC(10,2) NOT NULL,
  peso_desconto NUMERIC(10,2) DEFAULT 0,
```

```
    peso_liquido NUMERIC(10,2) GENERATED ALWAYS AS (peso_bruto - peso_desconto) STORED,  
    data_recebimento TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
-- Produção por Cooperado (Para cálculo de rateio)  
CREATE TABLE producao_cooperado (  
    id SERIAL PRIMARY KEY,  
    cooperado_id INT REFERENCES cooperados(id),  
    tipo_material VARCHAR(50) NOT NULL, -- Ex: PET, Papelão, Alumínio  
    peso_triado NUMERIC(10,2) NOT NULL,  
    data_triagem DATE DEFAULT CURRENT_DATE  
);
```

Vocês já utilizam algum sistema simples (como planilhas do Excel) para controlar a entrada de material e a produção dos cooperados hoje, ou querem desenhar o processo do zero?

User prompt: desenhar do zero!

Response: Perfeito! Desenhar do zero é a melhor oportunidade para criar um processo simples, eficiente e pensado exatamente para a rotina diária do barracão de triagem e da administração.

Para estruturar o ERP, precisamos organizar o sistema em **4 Grandes Pilares Operacionais**:

1. Fluxo Operacional e Módulos do ERP

[1. Entrada / Balança] → [2. Triagem e Processamento] → [3. Estoque e Venda] → [4. Financeiro e Rateio]

Módulo 1: Entrada e Recebimento de Materiais

- **Cadastro de Geradores/Fornecedores:** Empresas, condomínios, prefeitura ou catadores avulsos.
- **Pesagem de Entrada:** Registro de peso bruto, tara do veículo e peso líquido.
- **Classificação de Origem:** Identificar se o material foi doado, comprado ou recolhido via contrato público.

Módulo 2: Triagem e Produção (O Coração da Cooperativa)

- **Registro por Turno/Mesa:** Acompanhamento de quais cooperados trabalharam em determinada esteira/mesa de triagem.
- **Produção Final (Fardos/Lotes):** Apontamento de saída da triagem (ex: *Fardo nº 104 - PET Cristal - 120kg*).
- **Controle de Rejeito:** Registro do peso do material que não serve para reciclagem e vai para o aterro.

Módulo 3: Estoque e Comercialização

- **Gestão de Estoque Acabado:** Saldo atual de fardos e bag em estoque por tipo (Plástico, Papelão, Metais, Vidro).
- **Tabela de Preços de Venda:** Histórico de preços praticados por comprador/indústria.
- **Pedidos de Venda e Expedição:** Pesagem de saída na venda e emissão do ticket de saída.

Módulo 4: Gestão do Cooperado, Financeiro e Rateio

- **Frequência e Horas Trabalhadas:** Registro diário de presenças.
- **Cálculo de Rateio de Sobras:** Lógica parametrizável para divisão do resultado financeiro do mês (ex: baseado em horas trabalhadas ou quilos triados).
- **Descontos Fixo e Adiantamentos:** Adiantamentos da semana, custos com alimentação ou equipamentos.

2. Proposta de Arquitetura Técnica

Para manter o custo baixo ou zero de infraestrutura no início, a recomendação é usar tecnologias eficientes e de código aberto (*open-source*):

- **Front-end (Telas):** React (com Tailwind CSS) ou Flutter (caso queira apps para celular/tablet no barracão).
- **Back-end (Lógica e Regras):** Node.js (NestJS) ou Python (FastAPI).
- **Banco de Dados:** PostgreSQL.
- **Hospedagem Gratuita/Baixo Custo:** Render, Railway ou Supabase.

3. Próximos Passos do Nosso Projeto

Vamos avançar passo a passo para construir o blueprint completo do software.

Passo 1 (Próxima etapa): Definir a **Matriz de Regras de Negócio e Banco de Dados** completa. **Passo 2:** Desenhar as telas e o fluxo do usuário (UX/UI). **Passo 3:** Gerar o código do Back-end e Banco de Dados. **Passo 4:** Gerar o código das Telas da Aplicação.

Para começarmos o **Passo 1**, como é feita a divisão do dinheiro (rateio) na cooperativa de vocês hoje ou como pretendem fazer?

1. **Por Horas/Dias Trabalhados** (Todos ganham proporcional ao tempo presente na cooperativa).
2. **Por Produção Individual/Equipe** (Ganha proporcional ao quilo/fardo produzido).
3. **Misto** (Um valor fixo por hora + bônus de produção).

User prompt: *Dias Trabalhados (Todos ganham proporcional ao tempo presente na cooperativa).*

Response: Ótimo! O modelo por **Dias Trabalhados** é o mais utilizado em cooperativas de reciclagem, pois incentiva o trabalho em equipe na triagem e simplifica bastante a logística e o cálculo do rateio.

Com base nisso, vamos para o **Passo 1: Arquitetura do Banco de Dados e Regras do Rateio**.

Lógica do Rateio por Dias Trabalhados

No final do mês (ou período de fechamento), o cálculo financeiro funcionará da seguinte forma:

$$\text{Valor do Dia (Diária)} = \frac{\text{Receita Total de Vendas} - \text{Despesas Operacionais}}{\text{Total de Dias Trabalhados por TODOS os Cooperados}}$$

$$\text{Rateio do Cooperado} = (\text{Dias Trabalhados pelo Cooperado} \times \text{Valor do Dia}) - \text{Descontos/Adiantamentos}$$

Modelo do Banco de Dados (PostgreSQL)

Abaixo está o script SQL estruturado para suportar esse fluxo de trabalho e o cálculo automático do rateio:

```
-- 1. Cadastro de Cooperados
CREATE TABLE cooperados (
  id SERIAL PRIMARY KEY,
  nome VARCHAR(100) NOT NULL,
  cpf VARCHAR(14) UNIQUE NOT NULL,
  chave_pix VARCHAR(100),
  data_admissao DATE NOT NULL DEFAULT CURRENT_DATE,
  ativo BOOLEAN DEFAULT TRUE
);

-- 2. Registro Diário de Frequência (Presença)
CREATE TABLE presencas (
  id SERIAL PRIMARY KEY,
  cooperado_id INT REFERENCES cooperados(id),
  data DATE NOT NULL DEFAULT CURRENT_DATE,
  presente BOOLEAN DEFAULT TRUE,
  observacao TEXT,
  CONSTRAINT uniq_cooperado_dia UNIQUE (cooperado_id, data)
);

-- 3. Categorias de Materiais Recicláveis
CREATE TABLE tipos_material (
  id SERIAL PRIMARY KEY,
  nome VARCHAR(50) NOT NULL, -- Ex: PET Cristal, Papelão, Alumínio, Rejeito
  categoria VARCHAR(30) -- Plástico, Papel, Metal, Vidro, Aterro
);

-- 4. Vendas / Faturamento
CREATE TABLE vendas (
  id SERIAL PRIMARY KEY,
  comprador VARCHAR(100) NOT NULL,
  data_venda DATE DEFAULT CURRENT_DATE,
  valor_total NUMERIC(10,2) NOT NULL
);

-- 5. Itens da Venda (Detalhamento)
CREATE TABLE itens_venda (
  id SERIAL PRIMARY KEY,
  venda_id INT REFERENCES vendas(id) ON DELETE CASCADE,
  tipo_material_id INT REFERENCES tipos_material(id),
  peso_kg NUMERIC(10,2) NOT NULL,
  preco_por_kg NUMERIC(10,2) NOT NULL,
  subtotal NUMERIC(10,2) GENERATED ALWAYS AS (peso_kg * preco_por_kg) STORED
);

-- 6. Despesas Operacionais (Aluguel, Combustível, Manutenção, Aterro)
CREATE TABLE despesas (
  id SERIAL PRIMARY KEY,
  descricao VARCHAR(150) NOT NULL,
  valor NUMERIC(10,2) NOT NULL,
  data_despesa DATE DEFAULT CURRENT_DATE
);
```

```
-- 7. Adiantamentos e Descontos Individuais
CREATE TABLE adiantamentos_descontos (
  id SERIAL PRIMARY KEY,
  cooperado_id INT REFERENCES cooperados(id),
  descricao VARCHAR(100) NOT NULL, -- Ex: Vale, Compra de EPI
  valor NUMERIC(10,2) NOT NULL,
  data_registro DATE DEFAULT CURRENT_DATE,
  pago_ou_descontado BOOLEAN DEFAULT FALSE
);

-- 8. Fechamento de Rateio Mensal
CREATE TABLE fechamento_rateio (
  id SERIAL PRIMARY KEY,
  mes INT NOT NULL,
  ano INT NOT NULL,
  receita_bruta NUMERIC(10,2) NOT NULL,
  despesas_totais NUMERIC(10,2) NOT NULL,
  sobra_liquida NUMERIC(10,2) NOT NULL,
  total_dias_trabalhados_geral INT NOT NULL,
  valor_por_dia NUMERIC(10,2) NOT NULL,
  data_fechamento TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Próximo Passo: Telas da Aplicação (UX/UI)

Para facilitar a rotina no galpão e na gestão, precisaremos das seguintes telas principais:

1. **Chamada Diária (Tablet/Celular/PC):** Tela rápida para marcar presença dos cooperados do dia com 1 clique.
2. **Lançamento de Vendas e Pesagem:** Registro simples do peso dos fardos vendidos e valor por kg.
3. **Lançamento de Despesas e Vales:** Controle financeiro do barracão.
4. **Painel de Fechamento do Rateio:** Tela onde o gestor escolhe o mês/ano, o sistema calcula tudo automaticamente e gera a folha de pagamento de cada cooperado.

Podemos avançar para a criação dos endpoints do **Back-end** (códigos de API que calculam o rateio e gravam os dados) ou prefere ajustar alguma tabela do banco primeiro?

User prompt: vamos avançar!

Response: Avançando para o **Passo 2: Construção do Back-end**.

Vamos estruturar uma **API RESTful profissional usando Node.js (com Framework NestJS ou Express)**. O ponto central desta API é o serviço que executa a regra do cálculo financeiro e fecha a folha de pagamento do mês.

Lógica Principal da API: Serviço de Fechamento do Rateio

Abaixo está o código em JavaScript/TypeScript para a função que processa o rateio e gera o demonstrativo individual dos cooperados:

```
import { Client } from 'pg';

interface ItemRateioCooperado {
  cooperadoId: number;
  nome: number;
  diasTrabalhados: number;
  valorBruto: number;
  descontosAdiantamentos: number;
  valorLiquidoARreceber: number;
}

export async function processarFechamentoRateio(mes: number, ano: number, dbClient: Client) {
  // 1. Somar Receita Total de Vendas no Mês
  const resVendas = await dbClient.query(
    `SELECT COALESCE(SUM(valor_total), 0) as total
    FROM vendas
    WHERE EXTRACT(MONTH FROM data_venda) = $1 AND EXTRACT(YEAR FROM data_venda) = $2`,
    [mes, ano]
  );
  const receitaBruta = parseFloat(resVendas.rows[0].total);

  // 2. Somar Despesas Operacionais do Mês
  const resDespesas = await dbClient.query(
    `SELECT COALESCE(SUM(valor), 0) as total
    FROM despesas
    WHERE EXTRACT(MONTH FROM data_despesa) = $1 AND EXTRACT(YEAR FROM data_despesa) = $2`,
    [mes, ano]
  );
}
```

```

const despesasTotais = parseFloat(resDespesas.rows[0].total);

// 3. Sobre Líquida a ser distribuída
const sobraLiquida = receitaBruta - despesasTotais;

if (sobraLiquida <= 0) {
  throw new Error('A sobre líquida é menor ou igual a zero. Não há valor para rateio neste mês.');
```

}

// 4. Somar o TOTAL de dias trabalhados por TODOS os cooperados juntos no mês
const resDiasGeral = await dbClient.query(
 `SELECT COUNT(*) as total_dias
 FROM presencas
 WHERE presente = TRUE
 AND EXTRACT(MONTH FROM data) = \$1
 AND EXTRACT(YEAR FROM data) = \$2`,
 [mes, ano]
);
const totalDiasTrabalhadosGeral = parseInt(resDiasGeral.rows[0].total_dias, 10);

if (totalDiasTrabalhadosGeral === 0) {
 throw new Error('Nenhuma presença registrada para este período.');
}

// 5. Calcular o valor exato de 1 DIA TRABALHADO
const valorPorDia = sobraLiquida / totalDiasTrabalhadosGeral;

// 6. Buscar o total de dias trabalhados de CADA cooperado individualmente
const resCooperados = await dbClient.query(
 `SELECT c.id, c.nome, COUNT(p.id) as dias_trabalhados
 FROM cooperados c
 LEFT JOIN presencas p ON c.id = p.cooperado_id
 AND p.presente = TRUE
 AND EXTRACT(MONTH FROM p.data) = \$1
 AND EXTRACT(YEAR FROM p.data) = \$2
 WHERE c.ativo = TRUE
 GROUP BY c.id, c.nome`,
 [mes, ano]
);

const extratoCooperados: ItemRateioCooperado[] = [];

// 7. Calcular o valor individual subtraindo vales e adiantamentos
for (const row of resCooperados.rows) {
 const diasCoop = parseInt(row.dias_trabalhados, 10);
 const valorBrutoCoop = diasCoop * valorPorDia;

 // Buscar Vales/Descontos pendentes do cooperado no mês
 const resDescontos = await dbClient.query(
 `SELECT COALESCE(SUM(valor), 0) as total
 FROM adiantamentos_descontos
 WHERE cooperado_id = \$1
 AND pago_ou_descontado = FALSE
 AND EXTRACT(MONTH FROM data_registro) = \$2
 AND EXTRACT(YEAR FROM data_registro) = \$3`,
 [row.id, mes, ano]
);
 const descontos = parseFloat(resDescontos.rows[0].total);
 const valorLiquido = valorBrutoCoop - descontos;

 extratoCooperados.push({
 cooperadoId: row.id,
 nome: row.nome,
 diasTrabalhados: diasCoop,
 valorBruto: Math.max(0, valorBrutoCoop),
 descontosAdiantamentos: descontos,
 valorLiquidoARreceber: valorLiquido < 0 ? 0 : valorLiquido
 });
}

return {
 resumoGeral: {
 mes,
 ano,
 receitaBruta,
 despesasTotais,
 sobraLiquida,
 totalDiasTrabalhadosGeral,
 valorPorDia
 }
}

```
    },
    detalhamentoCooperados: extratoCooperados
  };
}
```

Endpoints da API (Rotas do Servidor)

Para disponibilizar essas funções para o Front-end, criaremos a seguinte estrutura de rotas:

Método	Rota	Descrição
POST	/api/cooperados	Cadastra novo cooperado
POST	/api/presencas/chamada-diaria	Salva a lista de presença do dia (lote)
POST	/api/vendas	Registra venda de fardos com peso e preço
POST	/api/despesas	Lança contas a pagar/pagas do galpão
POST	/api/adiantamentos	Lança vale/adiantamento para um cooperado
GET	/api/rateio/simulacao?mes=8&ano=2026	Retorna o pré-cálculo da folha antes de fechar
POST	/api/rateio/fechar	Grava o fechamento oficial do mês e marca vales como quitados

Próxima Etapa: Front-end (Telas do ERP)

Com a lógica do servidor pronta, o próximo passo é criar a interface gráfica para que a equipe da cooperativa possa usar no dia a dia.

User prompt: Vamos criar a tela de chamada diária de presenças para celular/tablet, com interface simples de marcar presente ou ausente.

Response: A tela de Chamada Diária para uso no galpão precisa ser ultra-simples, com botões grandes para toque em telas de celular ou tablet, evitando erros operacionais.

Abaixo está o código completo do componente em React + Tailwind CSS pronto para ser usado no seu aplicativo.

Código do Componente (ChamadaDiaria.jsx)

```
import React, { useState, useEffect } from 'react';
import { Check, X, Calendar, Save, UserCheck } from 'lucide-react';

export default function ChamadaDiaria() {
  // Data atual no formato YYYY-MM-DD para o input date
  const hoje = new Date().toISOString().split('T')[0];

  const [dataSelecionada, setDataSelecionada] = useState(hoje);
  const [carregando, setCarregando] = useState(false);
  const [mensagem, setMensagem] = useState('');

  // Lista mockada de cooperados (substituir pela chamada GET /api/cooperados)
  const [cooperados, setCooperados] = useState([
    { id: 1, nome: 'Ana Maria Silva', presente: true },
    { id: 2, nome: 'Carlos Eduardo Santos', presente: true },
    { id: 3, nome: 'João Pedro Oliveira', presente: false },
    { id: 4, nome: 'Maria Aparecida Costa', presente: true },
    { id: 5, nome: 'Roberto Souza', presente: true },
  ]);

  // Alterna o status de presença (Presente <-> Ausente)
  const alternarPresenca = (id) => {
    setCooperados(prev =>
      prev.map(c => (c.id === id ? { ...c, presente: !c.presente } : c))
    );
  };

  // Selecionar ou Desmarcar Todos
  const marcarTodos = (status) => {
    setCooperados(prev => prev.map(c => ({ ...c, presente: status })));
  };

  // Enviar a chamada para a API (POST /api/presencas/chamada-diaria)
  const salvarChamada = async () => {
    setCarregando(true);
    setMensagem('');

    const payload = {
      data: dataSelecionada,
```

```

    presencas: cooperados.map(c => ({
      cooperado_id: c.id,
      presente: c.presente
    })))
  });

  try {
    // Exemplo de integração com API:
    // await fetch('/api/presencas/chamada-diaria', {
    //   method: 'POST',
    //   headers: { 'Content-Type': 'application/json' },
    //   body: JSON.stringify(payload)
    // });

    console.log('Dados enviados:', payload);
    setMensagem('Chamada salva com sucesso!');
  } catch (error) {
    setMensagem('Erro ao salvar chamada.');
```

```

        {c.nome}
      </span>

      { /* Botão de Alternância */ }
      <div
        className={`w-12 h-12 rounded-full flex items-center justify-center transition-colors ${
          c.presente
            ? 'bg-emerald-500 text-white shadow-md'
            : 'bg-red-500 text-white'
        }`}
      >
        {c.presente ? <Check className="w-7 h-7" /> : <X className="w-7 h-7" />}
      </div>
    </div>
  )})
</div>

{ /* Mensagem de Feedback */ }
{mensagem && (
  <div className="mt-4 p-3 bg-emerald-100 border border-emerald-400 text-emerald-800 rounded-lg text-center fo
    {mensagem}
  </div>
)}

{ /* Botão Fixo de Salvar Rodapé */ }
<div className="fixed bottom-0 left-0 right-0 p-4 bg-white border-t border-gray-200 shadow-lg max-w-md mx-auto
  <button
    onClick={salvarChamada}
    disabled={carregando}
    className="w-full bg-emerald-600 hover:bg-emerald-700 active:bg-emerald-800 text-white font-bold py-3.5 px
  >
    <Save className="w-5 h-5" />
    {carregando ? 'Salvando...' : 'Confirmar Chamada'}
  </button>
</div>
</div>
);
}

```

Destaques da Interface

- **Touch Friendly:** Os cards inteiros são clicáveis com área de toque de **48px+**, facilitando o uso em smartphones e tablets no barracão.
- **Contador em Tempo Real:** Mostra instantaneamente quantos cooperados estão marcados como presentes do total.
- **Ações em Lote:** Permite "Marcar Todos" com apenas 1 toque no início do turno.
- **Seletor de Data:** Facilita correções retroativas caso a chamada não tenha sido lançada no próprio dia.

User prompt: Vamos desenhar o painel web de fechamento financeiro do mês, onde o gestor visualiza os totais e gera o rateio.

Response: O painel web de **Fechamento Financeiro e Rateio Mensal** é a tela administrativa onde o gestor analisa as receitas, despesas, valor da diária e gera o demonstrativo de pagamento de cada cooperado.

Geramos um exemplo em relatório PDF do painel para visualização técnica do layout:

[file-tag: code-generated-file-71ebae95-83f3-4046-bbc7-52be456f7483]

Código do Componente Web (PainelFechamentoRateio.jsx)

Abaixo está o código em **React + Tailwind CSS** pronto para compor o Dashboard Administrativo do seu ERP:

```

import React, { useState, useEffect } from 'react';
import {
  DollarSign,
  TrendingDown,
  TrendingUp,
  Calendar,
  Users,
  CheckCircle,
  AlertCircle,
  Download,
  Lock
} from 'lucide-react';

export default function PainelFechamentoRateio() {
  const [mes, setMes] = useState(8); // Agosto
  const [ano, setAno] = useState(2026);
  const [carregando, setCarregando] = useState(false);

```

```

const [fechado, setFechado] = useState(false);

// Estado dos dados simulados/retornados da API
const [dados, setDados] = useState({
  receitaBruta: 48500.00,
  despesasTotais: 12300.00,
  sobraLiquida: 36200.00,
  totalDiasGeral: 220,
  valorPorDia: 164.55,
  cooperados: [
    { id: 1, nome: 'Ana Maria Silva', dias: 22, bruto: 3620.10, vales: 200.00, liquido: 3420.10 },
    { id: 2, nome: 'Carlos Eduardo Santos', dias: 20, bruto: 3291.00, vales: 150.00, liquido: 3141.00 },
    { id: 3, nome: 'João Pedro Oliveira', dias: 18, bruto: 2961.90, vales: 300.00, liquido: 2661.90 },
    { id: 4, nome: 'Maria Aparecida Costa', dias: 22, bruto: 3620.10, vales: 100.00, liquido: 3520.10 },
    { id: 5, nome: 'Roberto Souza', dias: 21, bruto: 3455.55, vales: 250.00, liquido: 3205.55 },
    { id: 6, nome: 'Fernanda Lima', dias: 22, bruto: 3620.10, vales: 0.00, liquido: 3620.10 },
    { id: 7, nome: 'Gabriel Martins', dias: 19, bruto: 3126.45, vales: 120.00, liquido: 3006.45 },
  ]
});

// Função para simular ou buscar dados reais da API (GET /api/rateio/simulacao)
const buscarSimulacao = async () => {
  setCarregando(true);
  try {
    // Exemplo de integração real:
    // const res = await fetch(`/api/rateio/simulacao?mes=${mes}&ano=${ano}`);
    // const data = await res.json();
    // setDados(data);
  } catch (err) {
    console.error(err);
  } finally {
    setCarregando(false);
  }
};

// Função para efetivar o fechamento do mês (POST /api/rateio/fechar)
const executarFechamento = async () => {
  if (!window.confirm('Confirma o fechamento oficial do rateio de ${mes}/${ano}?')) return;

  setCarregando(true);
  try {
    // await fetch('/api/rateio/fechar', { method: 'POST', body: JSON.stringify({ mes, ano }) });
    setFechado(true);
  } catch (err) {
    alert('Erro ao fechar mês.');
  } finally {
    setCarregando(false);
  }
};

return (
  <div className="p-6 bg-slate-50 min-h-screen">
    /* Cabeçalho do Painel */
    <div className="flex flex-col md:flex-row md:items-center justify-between mb-6 gap-4">
      <div>
        <h1 className="text-2xl font-bold text-slate-800">Fechamento & Rateio Mensal</h1>
        <p className="text-slate-500 text-sm">Apuração de receitas, despesas e distribuição de sobras</p>
      </div>

      /* Filtro de Período */
      <div className="flex items-center gap-3 bg-white p-2 rounded-xl shadow-sm border border-slate-200">
        <Calendar className="w-5 h-5 text-emerald-600 ml-2" />
        <select
          value={mes}
          onChange={(e) => setMes(Number(e.target.value))}
          className="bg-transparent font-semibold text-slate-700 focus:outline-none"
        >
          <option value={1}>Janeiro</option>
          <option value={2}>Fevereiro</option>
          <option value={3}>Março</option>
          <option value={4}>Abril</option>
          <option value={5}>Maio</option>
          <option value={6}>Junho</option>
          <option value={7}>Julho</option>
          <option value={8}>Agosto</option>
          <option value={9}>Setembro</option>
          <option value={10}>Outubro</option>
          <option value={11}>Novembro</option>
          <option value={12}>Dezembro</option>
        </select>
      </div>
    </div>
  </div>
);

```

```

</select>
<select
  value={ano}
  onChange={(e) => setAno(Number(e.target.value))}
  className="bg-transparent font-semibold text-slate-700 focus:outline-none"
>
  <option value={2025}>2025</option>
  <option value={2026}>2026</option>
</select>
<button
  onClick={buscarSimulacao}
  className="bg-emerald-50 text-emerald-700 hover:bg-emerald-100 font-medium px-3 py-1.5 rounded-lg text-s
>
  Atualizar
</button>
</div>
</div>

{/* Cards de Métricas Financeiras */}
<div className="grid grid-cols-1 md:grid-cols-4 gap-4 mb-6">
  <div className="bg-white p-5 rounded-2xl border border-slate-200 shadow-sm">
    <div className="flex items-center justify-between text-slate-500 mb-2">
      <span className="text-xs font-bold uppercase tracking-wider">Vendas (Receita)</span>
      <div className="p-2 bg-emerald-50 rounded-lg text-emerald-600"><TrendingUp className="w-5 h-5" /></div>
    </div>
    <div className="text-2xl font-bold text-slate-900">R$ {dados.receitaBruta.toLocaleString('pt-BR', { minimu
</div>

  <div className="bg-white p-5 rounded-2xl border border-slate-200 shadow-sm">
    <div className="flex items-center justify-between text-slate-500 mb-2">
      <span className="text-xs font-bold uppercase tracking-wider">Despesas Operacionais</span>
      <div className="p-2 bg-rose-50 rounded-lg text-rose-600"><TrendingDown className="w-5 h-5" /></div>
    </div>
    <div className="text-2xl font-bold text-slate-900">R$ {dados.despesasTotais.toLocaleString('pt-BR', { mini
</div>

  <div className="bg-white p-5 rounded-2xl border border-slate-200 shadow-sm">
    <div className="flex items-center justify-between text-slate-500 mb-2">
      <span className="text-xs font-bold uppercase tracking-wider">Sobra Líquida</span>
      <div className="p-2 bg-blue-50 rounded-lg text-blue-600"><DollarSign className="w-5 h-5" /></div>
    </div>
    <div className="text-2xl font-bold text-blue-700">R$ {dados.sobraLiquida.toLocaleString('pt-BR', { minimum
</div>

  <div className="bg-emerald-900 text-white p-5 rounded-2xl shadow-sm">
    <div className="flex items-center justify-between text-emerald-200 mb-2">
      <span className="text-xs font-bold uppercase tracking-wider">Valor do Dia Trabalhado</span>
      <Users className="w-5 h-5 text-emerald-400" />
    </div>
    <div className="text-3xl font-extrabold text-emerald-400">
      R$ {dados.valorPorDia.toFixed(2)}
    </div>
    <div className="text-xs text-emerald-200 mt-1">
      Baseado em {dados.totalDiasGeral} diárias somadas
    </div>
  </div>
</div>

{/* Tabela do Demonstrativo de Rateio */}
<div className="bg-white rounded-2xl border border-slate-200 shadow-sm overflow-hidden mb-6">
  <div className="p-5 border-b border-slate-100 flex justify-between items-center">
    <h2 className="font-bold text-slate-800 text-lg">Demonstrativo Individual de Rateio</h2>
    <button className="flex items-center gap-2 text-slate-600 hover:text-emerald-700 font-medium text-sm">
      <Download className="w-4 h-4" /> Exportar Folha (PDF/Excel)
    </button>
  </div>

  <div className="overflow-x-auto">
    <table className="w-full text-left text-sm text-slate-600">
      <thead className="bg-slate-50 text-slate-500 uppercase text-xs font-semibold">
        <tr>
          <th className="p-4">Cooperado</th>
          <th className="p-4 text-center">Dias Trabalhados</th>
          <th className="p-4 text-right">Valor Bruto</th>
          <th className="p-4 text-right">Vales / Descontos</th>
          <th className="p-4 text-right">Líquido a Receber</th>
          <th className="p-4 text-center">Status</th>
        </tr>
      </thead>

```

```

<tbody className="divide-y divide-slate-100">
  {dados.cooperados.map((c) => {
    <tr key={c.id} className="hover:bg-slate-50 transition-colors">
      <td className="p-4 font-semibold text-slate-800">{c.nome}</td>
      <td className="p-4 text-center font-medium">{c.dias} dias</td>
      <td className="p-4 text-right text-slate-600">R$ {c.bruto.toFixed(2)}</td>
      <td className="p-4 text-right text-rose-600">R$ {c.vales.toFixed(2)}</td>
      <td className="p-4 text-right font-bold text-emerald-700 text-base">
        R$ {c.liquido.toFixed(2)}
      </td>
      <td className="p-4 text-center">
        <span className="inline-flex items-center gap-1 bg-emerald-100 text-emerald-800 text-xs px-2.5 p-1">
          <CheckCircle className="w-3.5 h-3.5" /> Calculado
        </span>
      </td>
    </tr>
  })}
</tbody>
</table>
</div>
</div>

{/* Ação de Fechamento */}
<div className="flex justify-end gap-4">
  <button
    onClick={executarFechamento}
    disabled={fechado || carregando}
    className={`flex items-center gap-2 font-bold py-3.5 px-6 rounded-xl shadow-md transition-all ${
      fechado
        ? 'bg-slate-300 text-slate-600 cursor-not-allowed'
        : 'bg-emerald-600 hover:bg-emerald-700 text-white active:scale-[0.98]'
    }`}
  >
    <Lock className="w-5 h-5" />
    {fechado ? 'Mês Fechado e Homologado' : 'Efetivar e Fechar Mês'}
  </button>
</div>
</div>
);
}

```

User prompt: Vamos criar a tela de lançamento de vendas de fardos e pesagens para registrar a receita do galpão.

Response: A tela de **Lançamento de Vendas e Pesagem** é fundamental para alimentar a **Receita Bruta** no sistema. Ela permite selecionar o comprador, adicionar múltiplos itens (PET, Papelão, Alumínio, etc.) com o peso em kg e valor por kg, calculando o total da nota automaticamente.

Código do Componente (LancamentoVendas.jsx)

```

import React, { useState } from 'react';
import { ShoppingCart, Scale, Plus, Trash2, Save, FileText, DollarSign } from 'lucide-react';

export default function LancamentoVendas() {
  const [comprador, setComprador] = useState('');
  const [dataVenda, setDataVenda] = useState(new Date().toISOString().split('T')[0]);
  const [carregando, setCarregando] = useState(false);
  const [sucesso, setSucesso] = useState(false);

  // Lista de materiais cadastrados (para o select)
  const tiposMaterial = [
    { id: 1, nome: 'PET Cristal (Fardo)' },
    { id: 2, nome: 'PET Colorido (Fardo)' },
    { id: 3, nome: 'Papelão Ondulado' },
    { id: 4, nome: 'Alumínio (Lata)' },
    { id: 5, nome: 'PEAD (Plástico Duro)' },
    { id: 6, nome: 'Vidro Moído' }
  ];

  // Item em digitação na linha de inserção rápida
  const [novoItem, setNovoItem] = useState({
    tipo_material_id: 1,
    peso_kg: '',
    preco_por_kg: ''
  });

  // Lista de itens adicionados à venda atual
  const [itensVenda, setItensVenda] = useState([
    { id: 1, tipo_material_id: 1, nome: 'PET Cristal (Fardo)', peso_kg: 450, preco_por_kg: 3.20, subtotal: 1440.00 }
  ])

```

```

    { id: 2, tipo_material_id: 3, nome: 'Papelo Ondulado', peso_kg: 1200, preco_por_kg: 0.65, subtotal: 780.00 }
  });

// Adicionar item à tabela
const adicionarItem = () => {
  if (!novoItem.peso_kg || !novoItem.preco_por_kg) return;

  const mat = tiposMaterial.find(m => m.id === Number(novoItem.tipo_material_id));
  const peso = parseFloat(novoItem.peso_kg);
  const preco = parseFloat(novoItem.preco_por_kg);
  const subtotal = peso * preco;

  const itemFormatado = {
    id: Date.now(),
    tipo_material_id: mat.id,
    nome: mat.nome,
    peso_kg: peso,
    preco_por_kg: preco,
    subtotal
  };

  setItensVenda([...itensVenda, itemFormatado]);
  setNovoItem({ tipo_material_id: 1, peso_kg: '', preco_por_kg: '' });
};

// Remover item da tabela
const removerItem = (id) => {
  setItensVenda(itensVenda.filter(item => item.id !== id));
};

// Totais da Nota de Venda
const pesoTotalKg = itensVenda.reduce((acc, item) => acc + item.peso_kg, 0);
const valorTotalVenda = itensVenda.reduce((acc, item) => acc + item.subtotal, 0);

// Enviar para o Back-end (POST /api/vendas)
const salvarVenda = async (e) => {
  e.preventDefault();
  if (!comprador || itensVenda.length === 0) {
    alert('Informe o comprador e adicione ao menos um item.');
```

```

    <p className="text-slate-500 text-sm">Registre a saída de fardos e faturamento para os compradores</p>
  </div>
</div>

{/* Formulário Principal */}
<form onSubmit={salvarVenda} className="space-y-6">

  {/* Dados do Comprador e Data */}
  <div className="bg-white p-5 rounded-2xl border border-slate-200 shadow-sm grid grid-cols-1 md:grid-cols-3">
    <div className="md:col-span-2">
      <label className="block text-xs font-bold text-slate-600 uppercase mb-1">
        Empresa Compradora / Indústria
      </label>
      <input
        type="text"
        placeholder="Ex: Reciclagem Indústria X LTDA"
        value={comprador}
        onChange={(e) => setComprador(e.target.value)}
        required
        className="w-full p-3 border border-slate-300 rounded-xl focus:ring-2 focus:ring-emerald-500 focus:outline-none"
      />
    </div>

    <div>
      <label className="block text-xs font-bold text-slate-600 uppercase mb-1">
        Data da Saída
      </label>
      <input
        type="date"
        value={dataVenda}
        onChange={(e) => setDataVenda(e.target.value)}
        required
        className="w-full p-3 border border-slate-300 rounded-xl focus:ring-2 focus:ring-emerald-500 focus:outline-none"
      />
    </div>
  </div>

  {/* Adicionar Itens / Pesagem */}
  <div className="bg-white p-5 rounded-2xl border border-slate-200 shadow-sm">
    <h2 className="text-sm font-bold text-slate-700 uppercase mb-3 flex items-center gap-2">
      <Scale className="w-4 h-4 text-emerald-600" />
      Adicionar Materiais Pesados
    </h2>

    <div className="grid grid-cols-1 md:grid-cols-12 gap-3 mb-4">
      <div className="md:col-span-5">
        <select
          value={novoItem.tipo_material_id}
          onChange={(e) => setNovoItem({ ...novoItem, tipo_material_id: e.target.value })}
          className="w-full p-3 border border-slate-300 rounded-xl text-slate-700 focus:ring-2 focus:ring-emerald-500"
        >
          {tiposMaterial.map(m => (
            <option key={m.id} value={m.id}>{m.nome}</option>
          ))}
        </select>
      </div>

      <div className="md:col-span-3">
        <input
          type="number"
          step="0.01"
          placeholder="Peso (kg)"
          value={novoItem.peso_kg}
          onChange={(e) => setNovoItem({ ...novoItem, peso_kg: e.target.value })}
          className="w-full p-3 border border-slate-300 rounded-xl focus:ring-2 focus:ring-emerald-500"
        />
      </div>

      <div className="md:col-span-3">
        <input
          type="number"
          step="0.01"
          placeholder="Preço/kg (R$)"
          value={novoItem.preco_por_kg}
          onChange={(e) => setNovoItem({ ...novoItem, preco_por_kg: e.target.value })}
          className="w-full p-3 border border-slate-300 rounded-xl focus:ring-2 focus:ring-emerald-500"
        />
      </div>
    </div>
  </div>

```

```

<div className="md:col-span-1">
  <button
    type="button"
    onClick={adicionarItem}
    className="w-full h-full bg-emerald-600 hover:bg-emerald-700 text-white rounded-xl flex items-cent
    title="Adicionar Item"
  >
    <Plus className="w-6 h-6" />
  </button>
</div>
</div>

{/* Tabela de Itens Adicionados */}
<div className="overflow-x-auto border-t border-slate-100 pt-3">
  <table className="w-full text-left text-sm text-slate-600">
    <thead className="bg-slate-50 text-slate-500 uppercase text-xs">
      <tr>
        <th className="p-3">Material</th>
        <th className="p-3 text-right">Peso Total</th>
        <th className="p-3 text-right">Valor / kg</th>
        <th className="p-3 text-right">Subtotal</th>
        <th className="p-3 text-center">Ação</th>
      </tr>
    </thead>
    <tbody className="divide-y divide-slate-100">
      {itensVenda.map((item) => (
        <tr key={item.id}>
          <td className="p-3 font-semibold text-slate-800">{item.nome}</td>
          <td className="p-3 text-right font-medium">{item.peso_kg.toLocaleString('pt-BR')} kg</td>
          <td className="p-3 text-right">R$ {item.preco_por_kg.toFixed(2)}</td>
          <td className="p-3 text-right font-bold text-slate-900">R$ {item.subtotal.toFixed(2)}</td>
          <td className="p-3 text-center">
            <button
              type="button"
              onClick={() => removerItem(item.id)}
              className="text-rose-500 hover:text-rose-700 p-1 rounded-lg"
            >
              <Trash2 className="w-4 h-4" />
            </button>
          </td>
        </tr>
      ))}
      {itensVenda.length === 0 && (
        <tr>
          <td colSpan="5" className="p-4 text-center text-slate-400 italic">
            Nenhum material adicionado a esta venda.
          </td>
        </tr>
      )}
    </tbody>
  </table>
</div>
</div>

{/* Card com Resumo Totais e Botão Salvar */}
<div className="bg-slate-900 text-white p-5 rounded-2xl flex flex-col md:flex-row justify-between items-ce
  <div className="flex gap-8">
    <div>
      <span className="text-xs text-slate-400 uppercase font-bold block">Peso Total da Carga</span>
      <span className="text-xl font-bold text-slate-100">{pesoTotalKg.toLocaleString('pt-BR')} kg</span>
    </div>
    <div>
      <span className="text-xs text-emerald-400 uppercase font-bold block">Valor Total da Venda</span>
      <span className="text-2xl font-extrabold text-emerald-400">R$ {valorTotalVenda.toFixed(2)}</span>
    </div>
  </div>

  <button
    type="submit"
    disabled={carregando || itensVenda.length === 0}
    className="w-full md:w-auto bg-emerald-500 hover:bg-emerald-600 active:bg-emerald-700 font-bold px-8 p
  >
    <Save className="w-5 h-5" />
    {carregando ? 'Registrando...' : 'Finalizar Venda'}
  </button>
</div>

{sucesso && (
  <div className="p-4 bg-emerald-100 border border-emerald-400 text-emerald-800 rounded-xl text-center fon

```

```

        Venda registrada com sucesso no caixa do galpão!
      </div>
    }}
  </form>
</div>
</div>
);
}

```

User prompt: Vamos criar a tela para controle de despesas operacionais e lançamento de vales/adiantamentos para cooperados.

Response: A tela de **Gestão Financeira (Despesas e Vales)** é dividida em duas abas para simplificar o uso: uma para as contas operacionais do galpão (aluguel, combustível, manutenção da prensa) e outra para os adiantamentos concedidos aos cooperados.

Código do Componente (GestaoFinanceira.jsx)

```

import React, { useState } from 'react';
import { CreditCard, DollarSign, UserCheck, Plus, Trash2, Calendar, FileText, CheckCircle2 } from 'lucide-react';

export default function GestaoFinanceira() {
  const [abaAtiva, setAbaAtiva] = useState('despesas'); // 'despesas' ou 'vales'

  // Lista mockada de cooperados ativos
  const cooperados = [
    { id: 1, nome: 'Ana Maria Silva' },
    { id: 2, nome: 'Carlos Eduardo Santos' },
    { id: 3, nome: 'João Pedro Oliveira' },
    { id: 4, nome: 'Maria Aparecida Costa' },
    { id: 5, nome: 'Roberto Souza' },
  ];

  // Estado das Despesas
  const [despesas, setDespesas] = useState([
    { id: 1, descricao: 'Manutenção da Prensa Hidráulica', valor: 850.00, data: '2026-08-10' },
    { id: 2, descricao: 'Combustível do Caminhão de Coleta', valor: 420.00, data: '2026-08-15' },
    { id: 3, descricao: 'Energia Elétrica do Galpão', valor: 1250.00, data: '2026-08-18' },
  ]);
  const [novaDespesa, setNovaDespesa] = useState({ descricao: '', valor: '', data: new Date().toISOString().split('T')[0] });

  // Estado dos Vales/Adiantamentos
  const [vales, setVales] = useState([
    { id: 1, cooperado_id: 1, nome: 'Ana Maria Silva', descricao: 'Vale Mercado', valor: 200.00, data: '2026-08-05' },
    { id: 2, cooperado_id: 3, nome: 'João Pedro Oliveira', descricao: 'Adiantamento Emergencial', valor: 300.00, data: '2026-08-12' },
  ]);
  const [novoVale, setNovoVale] = useState({ cooperado_id: 1, descricao: '', valor: '', data: new Date().toISOString().split('T')[0] });

  // Adicionar Despesa Operacional
  const adicionarDespesa = (e) => {
    e.preventDefault();
    if (!novaDespesa.descricao || !novaDespesa.valor) return;

    setDespesas([...despesas, {
      id: Date.now(),
      descricao: novaDespesa.descricao,
      valor: parseFloat(novaDespesa.valor),
      data: novaDespesa.data
    }]);
    setNovaDespesa({ descricao: '', valor: '', data: new Date().toISOString().split('T')[0] });
  };

  // Adicionar Vale a Cooperado
  const adicionarVale = (e) => {
    e.preventDefault();
    if (!novoVale.descricao || !novoVale.valor) return;

    const coop = cooperados.find(c => c.id === Number(novoVale.cooperado_id));

    setVales([...vales, {
      id: Date.now(),
      cooperado_id: coop.id,
      nome: coop.nome,
      descricao: novoVale.descricao,
      valor: parseFloat(novoVale.valor),
      data: novoVale.data
    }]);
    setNovoVale({ cooperado_id: 1, descricao: '', valor: '', data: new Date().toISOString().split('T')[0] });
  };

```

```

};

const totalDespesas = despesas.reduce((acc, d) => acc + d.valor, 0);
const totalVales = vales.reduce((acc, v) => acc + v.valor, 0);

return (
  <div className="p-6 bg-slate-50 min-h-screen">
    <div className="max-w-5xl mx-auto">

      {/* Cabeçalho */}
      <div className="flex flex-col md:flex-row md:items-center justify-between mb-6 gap-4">
        <div>
          <h1 className="text-2xl font-bold text-slate-800 flex items-center gap-2">
            <CreditCard className="w-7 h-7 text-emerald-600" />
            Gestão Financeira & Vales
          </h1>
          <p className="text-slate-500 text-sm">Lançamento de custos operacionais e adiantamentos de cooperados</p>
        </div>

        {/* Navegação entre Abas */}
        <div className="flex bg-slate-200 p-1 rounded-xl">
          <button
            onClick={() => setAbaAtiva('despesas')}
            className={`px-4 py-2 rounded-lg font-bold text-sm transition-all ${
              abaAtiva === 'despesas' ? 'bg-white text-slate-800 shadow-sm' : 'text-slate-600 hover:text-slate-900'
            }`}
          >
            Despesas Operacionais
          </button>
          <button
            onClick={() => setAbaAtiva('vales')}
            className={`px-4 py-2 rounded-lg font-bold text-sm transition-all ${
              abaAtiva === 'vales' ? 'bg-white text-slate-800 shadow-sm' : 'text-slate-600 hover:text-slate-900'
            }`}
          >
            Vales & Adiantamentos
          </button>
        </div>
      </div>

      {/* ABA 1: DESPESAS OPERACIONAIS */}
      {abaAtiva === 'despesas' && (
        <div className="space-y-6">

          {/* Formulário de Despesa */}
          <form onSubmit={adicionarDespesa} className="bg-white p-5 rounded-2xl border border-slate-200 shadow-sm">
            <h2 className="text-sm font-bold text-slate-700 uppercase mb-3 flex items-center gap-2">
              <Plus className="w-4 h-4 text-emerald-600" /> Nova Despesa Operacional
            </h2>
            <div className="grid grid-cols-1 md:grid-cols-12 gap-3">
              <div className="md:col-span-6">
                <input
                  type="text"
                  placeholder="Descrição (ex: Manutenção Prensa, Combustível)"
                  value={novaDespesa.descricao}
                  onChange={(e) => setNovaDespesa({ ...novaDespesa, descricao: e.target.value })}
                  required
                  className="w-full p-3 border border-slate-300 rounded-xl focus:ring-2 focus:ring-emerald-500"
                />
              </div>
              <div className="md:col-span-3">
                <input
                  type="number"
                  step="0.01"
                  placeholder="Valor (R$)"
                  value={novaDespesa.valor}
                  onChange={(e) => setNovaDespesa({ ...novaDespesa, valor: e.target.value })}
                  required
                  className="w-full p-3 border border-slate-300 rounded-xl focus:ring-2 focus:ring-emerald-500"
                />
              </div>
              <div className="md:col-span-3">
                <input
                  type="date"
                  value={novaDespesa.data}
                  onChange={(e) => setNovaDespesa({ ...novaDespesa, data: e.target.value })}
                  required
                  className="w-full p-3 border border-slate-300 rounded-xl focus:ring-2 focus:ring-emerald-500"
                />
              </div>
            </div>
          </form>
        </div>
      )}
    </div>
  </div>
);

```

```

    </div>
  </div>
  <button
    type="submit"
    className="mt-3 w-full bg-emerald-600 hover:bg-emerald-700 text-white font-bold py-3 rounded-xl tran
  >
    Lançar Despesa
  </button>
</form>

{/* Lista de Despesas */}
<div className="bg-white rounded-2xl border border-slate-200 shadow-sm overflow-hidden">
  <div className="p-4 bg-slate-50 border-b border-slate-100 flex justify-between items-center">
    <span className="font-bold text-slate-700 text-sm uppercase">Despesas do Mês Atual</span>
    <span className="font-extrabold text-rose-600 text-base">Total: R$ {totalDespesas.toFixed(2)}</span>
  </div>
  <table className="w-full text-left text-sm text-slate-600">
    <thead className="bg-slate-50 text-slate-500 uppercase text-xs">
      <tr>
        <th className="p-3">Data</th>
        <th className="p-3">Descrição</th>
        <th className="p-3 text-right">Valor</th>
        <th className="p-3 text-center">Ação</th>
      </tr>
    </thead>
    <tbody className="divide-y divide-slate-100">
      {despesas.map((d) => (
        <tr key={d.id}>
          <td className="p-3 font-medium">{d.data}</td>
          <td className="p-3 font-semibold text-slate-800">{d.descricao}</td>
          <td className="p-3 text-right font-bold text-rose-600">R$ {d.valor.toFixed(2)}</td>
          <td className="p-3 text-center">
            <button onClick={() => setDespesas(despesas.filter(item => item.id !== d.id))} className="te
              <Trash2 className="w-4 h-4" />
            </button>
          </td>
        </tr>
      ))}
    </tbody>
  </table>
</div>

</div>
)}}

{/* ABA 2: VALES E ADIANTAMENTOS */}
{abaAtiva === 'vales' && (
  <div className="space-y-6">

    {/* Formulário de Vales */}
    <form onSubmit={adicionarVale} className="bg-white p-5 rounded-2xl border border-slate-200 shadow-sm">
      <h2 className="text-sm font-bold text-slate-700 uppercase mb-3 flex items-center gap-2">
        <Plus className="w-4 h-4 text-emerald-600" /> Lançar Vale / Adiantamento
      </h2>
      <div className="grid grid-cols-1 md:grid-cols-12 gap-3">
        <div className="md:col-span-4">
          <select
            value={novoVale.cooperado_id}
            onChange={(e) => setNovoVale({ ...novoVale, cooperado_id: e.target.value })}
            className="w-full p-3 border border-slate-300 rounded-xl focus:ring-2 focus:ring-emerald-500"
          >
            {cooperados.map(c => (
              <option key={c.id} value={c.id}>{c.nome}</option>
            ))}
          </select>
        </div>
        <div className="md:col-span-4">
          <input
            type="text"
            placeholder="Descrição (ex: Vale Alimentação, Farmácia)"
            value={novoVale.descricao}
            onChange={(e) => setNovoVale({ ...novoVale, descricao: e.target.value })}
            required
            className="w-full p-3 border border-slate-300 rounded-xl focus:ring-2 focus:ring-emerald-500"
          />
        </div>
        <div className="md:col-span-2">
          <input
            type="number"

```

```

        step="0.01"
        placeholder="Valor (R$)"
        value={novoVale.valor}
        onChange={(e) => setNovoVale({ ...novoVale, valor: e.target.value })}
        required
        className="w-full p-3 border border-slate-300 rounded-xl focus:ring-2 focus:ring-emerald-500"
      />
    </div>
    <div className="md:col-span-2">
      <input
        type="date"
        value={novoVale.data}
        onChange={(e) => setNovoVale({ ...novoVale, data: e.target.value })}
        required
        className="w-full p-3 border border-slate-300 rounded-xl focus:ring-2 focus:ring-emerald-500"
      />
    </div>
  </div>
  <button
    type="submit"
    className="mt-3 w-full bg-emerald-600 hover:bg-emerald-700 text-white font-bold py-3 rounded-xl tran
  >
    Confirmar Vale
  </button>
</form>

{/* Lista de Vales */}
<div className="bg-white rounded-2xl border border-slate-200 shadow-sm overflow-hidden">
  <div className="p-4 bg-slate-50 border-b border-slate-100 flex justify-between items-center">
    <span className="font-bold text-slate-700 text-sm uppercase">Vales Pendentes de Desconto no Rateio</span>
    <span className="font-extrabold text-amber-600 text-base">Total a Descontar: R$ {totalVales.toFixed(
  </div>
  <table className="w-full text-left text-sm text-slate-600">
    <thead className="bg-slate-50 text-slate-500 uppercase text-xs">
      <tr>
        <th className="p-3">Data</th>
        <th className="p-3">Cooperado</th>
        <th className="p-3">Motivo/Descrição</th>
        <th className="p-3 text-right">Valor Desconto</th>
        <th className="p-3 text-center">Ação</th>
      </tr>
    </thead>
    <tbody className="divide-y divide-slate-100">
      {vales.map((v) => (
        <tr key={v.id}>
          <td className="p-3 font-medium">{v.data}</td>
          <td className="p-3 font-bold text-slate-800">{v.nome}</td>
          <td className="p-3">{v.descricao}</td>
          <td className="p-3 text-right font-bold text-amber-600">R$ {v.valor.toFixed(2)}</td>
          <td className="p-3 text-center">
            <button onClick={() => setVales(vales.filter(item => item.id !== v.id))} className="text-ros
              <Trash2 className="w-4 h-4" />
            </button>
          </td>
        </tr>
      ))}
    </tbody>
  </table>
</div>
</div>
)}
</div>
</div>
);
}

```

User prompt: Gerar a estrutura de pastas e arquivos para começarmos a juntar tudo no projeto do ERP.

Response: Esta é a estrutura de pastas organizada para a arquitetura completa do **ERP de Reciclagem**, separando o **Back-end** (API Node.js) e o **Front-end** (React + Tailwind CSS).

Visão Geral da Estrutura

```

erp-cooperativa/
|

```

```

├── backend/                                # Servidor API Node.js (Express / NestJS)
│   ├── src/
│   │   ├── config/                        # Conexão com banco de dados e variáveis de ambiente
│   │   │   ├── database.js               # Script de conexão Pool do PostgreSQL
│   │   ├── controllers/                  # Regras de transporte (Lida com Requests e Responses)
│   │   │   ├── cooperados.controller.js
│   │   │   ├── presencas.controller.js
│   │   │   ├── vendas.controller.js
│   │   │   ├── despesas.controller.js
│   │   │   └── rateio.controller.js
│   │   ├── services/                    # Regras de Negócio e Cálculos Financeiros
│   │   │   └── rateio.service.js         # Algoritmo de cálculo por diárias
│   │   ├── routes/                      # Mapeamento de rotas HTTP
│   │   │   ├── cooperados.routes.js
│   │   │   ├── presencas.routes.js
│   │   │   ├── vendas.routes.js
│   │   │   ├── despesas.routes.js
│   │   │   └── rateio.routes.js
│   │   └── server.js                    # Ponto de entrada (Boot da aplicação)
│   ├── .env                             # Variáveis de ambiente (DATABASE_URL, PORT)
│   ├── init.sql                          # Script SQL com as tabelas criadas
│   └── package.json
└── frontend/                             # Interface do Usuário (React + Vite + Tailwind)
    ├── src/
    │   ├── components/                  # Telas e interfaces construídas
    │   │   ├── ChamadaDiaria.jsx        # Tela mobile para o galpão
    │   │   ├── PainelFechamentoRateio.jsx # Dashboard financeiro do gestor
    │   │   ├── LancamentoVendas.jsx     # Pesagem e vendas de fardos
    │   │   └── GestaoFinanceira.jsx      # Despesas e vales dos cooperados
    │   ├── services/                   # Comunicação HTTP com o Back-end
    │   │   └── api.js                  # Configuração do Axios ou Fetch API
    │   ├── App.jsx                     # Componente principal / Roteamento de telas
    │   ├── main.jsx                    # Renderizador do React
    │   ├── index.css                   # Configuração e diretivas do Tailwind CSS
    └── tailwind.config.js
    └── package.json

```

Arquivos de Conexão Rápida

backend/src/config/database.js

```

const { Pool } = require('pg');

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: process.env.NODE_ENV === 'production' ? { rejectUnauthorized: false } : false
});

module.exports = pool;

```

frontend/src/services/api.js

```

const API_URL = import.meta.env.VITE_API_URL || 'http://localhost:3000/api';

export async function fetchApi(endpoint, options = {}) {
  const response = await fetch(`${API_URL}${endpoint}`, {
    headers: { 'Content-Type': 'application/json', ...options.headers },
    ...options
  });
  if (!response.ok) throw new Error('Erro na requisição');
  return response.json();
}

```

User prompt: Vamos criar o código do arquivo server.js e a estrutura de rotas do Node.js.

Response: O servidor Node.js organiza a comunicação do banco de dados com as rotas HTTP e aplica as regras do ERP.

1. Ponto de Entrada (backend/src/server.js)

```

require('dotenv').config();
const express = require('express');
const cors = require('cors');

const cooperadosRoutes = require('./routes/cooperados.routes');
const presencasRoutes = require('./routes/presencas.routes');
const vendasRoutes = require('./routes/vendas.routes');
const despesasRoutes = require('./routes/despesas.routes');

```

```
const rateioRoutes = require('./routes/rateio.routes');

const app = express();

// Middlewares
app.use(cors());
app.use(express.json());

// Registro de Rotas da API
app.use('/api/cooperados', cooperadosRoutes);
app.use('/api/presencas', presencasRoutes);
app.use('/api/vendas', vendasRoutes);
app.use('/api/despesas', despesasRoutes);
app.use('/api/rateio', rateioRoutes);

// Rota de Healthcheck
app.get('/health', (req, res) => {
  res.json({ status: 'ok', timestamp: new Date() });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`🚀 Servidor ERP rodando na porta ${PORT}`);
});
```

2. Rotas Principais do Node.js

backend/src/routes/presencas.routes.js (Chamada Diária)

```
const express = require('express');
const router = express.Router();
const pool = require('../../config/database');

// Registrar ou atualizar lista de presença (Lote)
router.post('/chamada-diaria', async (req, res) => {
  const client = await pool.connect();
  try {
    const { data, presencas } = req.body; // presencas: [{ cooperado_id, presente }]
    await client.query('BEGIN');

    for (const item of presencas) {
      await client.query(
        `INSERT INTO presencas (cooperado_id, data, presente)
        VALUES ($1, $2, $3)
        ON CONFLICT (cooperado_id, data)
        DO UPDATE SET presente = EXCLUDED.presente`,
        [item.cooperado_id, data, item.presente]
      );
    }

    await client.query('COMMIT');
    res.json({ message: 'Chamada registrada com sucesso!' });
  } catch (err) {
    await client.query('ROLLBACK');
    res.status(500).json({ error: 'Erro ao salvar chamada', details: err.message });
  } finally {
    client.release();
  }
});

module.exports = router;
```

backend/src/routes/vendas.routes.js (Lançamento de Vendas)

```
const express = require('express');
const router = express.Router();
const pool = require('../../config/database');

// Lançar Venda com Itens (Transação)
router.post('/', async (req, res) => {
  const client = await pool.connect();
  try {
    const { comprador, data_venda, valor_total, itens } = req.body;
    await client.query('BEGIN');

    // 1. Inserir cabeçalho da venda
    const resVenda = await client.query(
      `INSERT INTO vendas (comprador, data_venda, valor_total)
      VALUES ($1, $2, $3) RETURNING id`,
      [comprador, data_venda, valor_total]
    );
  } catch (err) {
    await client.query('ROLLBACK');
    res.status(500).json({ error: 'Erro ao lançar venda', details: err.message });
  } finally {
    client.release();
  }
});
```

```

    [comprador, data_venda, valor_total]
  );
  const vendaId = resVenda.rows[0].id;

  // 2. Inserir itens pesados
  for (const item of itens) {
    await client.query(
      `INSERT INTO itens_venda (venda_id, tipo_material_id, peso_kg, preco_por_kg)
      VALUES ($1, $2, $3, $4)`,
      [vendaId, item.tipo_material_id, item.peso_kg, item.preco_por_kg]
    );
  }

  await client.query('COMMIT');
  res.status(201).json({ id: vendaId, message: 'Venda e pesagem registradas!' });
} catch (err) {
  await client.query('ROLLBACK');
  res.status(500).json({ error: 'Erro ao registrar venda', details: err.message });
} finally {
  client.release();
}
});

module.exports = router;

```

backend/src/routes/rateio.routes.js (Simulação e Fechamento)

```

const express = require('express');
const router = express.Router();
const pool = require('../config/database');
const { processarFechamentoRateio } = require('../services/rateio.service');

// Consultar simulação do mês
router.get('/simulacao', async (req, res) => {
  try {
    const { mes, ano } = req.query;
    const resultado = await processarFechamentoRateio(Number(mes), Number(ano), pool);
    res.json(resultado);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});

module.exports = router;

```

User prompt: Vamos criar o App.jsx no React com o menu lateral para alternar entre todas as telas que desenvolvemos.

Response: O App.jsx funciona como o hub principal da aplicação React, oferecendo uma barra lateral (Sidebar) expansível e responsiva para navegar entre os 4 módulos operacionais do ERP de reciclagem.

Código do Componente Principal (frontend/src/App.jsx)

```

import React, { useState } from 'react';
import {
  UserCheck,
  DollarSign,
  ShoppingCart,
  CreditCard,
  Recycle,
  Menu,
  X
} from 'lucide-react';

// Importação das telas desenvolvidas
import ChamadaDiaria from './components/ChamadaDiaria';
import PainelFechamentoRateio from './components/PainelFechamentoRateio';
import LancamentoVendas from './components/LancamentoVendas';
import GestaoFinanceira from './components/GestaoFinanceira';

export default function App() {
  const [telaAtiva, setTelaAtiva] = useState('chamada');
  const [menuAberto, setMenuAberto] = useState(false);

  // Mapeamento dos itens do menu lateral
  const menus = [
    { id: 'chamada', rotulo: 'Chamada Diária', icone: UserCheck },
    { id: 'vendas', rotulo: 'Vendas & Pesagem', icone: ShoppingCart },
    { id: 'financeiro', rotulo: 'Despesas & Vales', icone: CreditCard },
  ]

```

```

    { id: 'rateio', rotulo: 'Fechamento & Rateio', icone: DollarSign },
  ];

  // Renderização condicional da tela ativa
  const renderizarTela = () => {
    switch (telaAtiva) {
      case 'chamada': return <ChamadaDiaria />;
      case 'vendas': return <LancamentoVendas />;
      case 'financeiro': return <GestaoFinanceira />;
      case 'rateio': return <PainelFechamentoRateio />;
      default: return <ChamadaDiaria />;
    }
  };

  return (
    <div className="flex h-screen bg-slate-100 font-sans overflow-hidden">

      {/* Sidebar - Desktop */}
      <aside className="hidden md:flex flex-col w-64 bg-slate-900 text-white shadow-xl">
        {/* Banner do App */}
        <div className="p-5 border-b border-slate-800 flex items-center gap-3">
          <div className="p-2 bg-emerald-500 rounded-xl text-white">
            <Recycle className="w-6 h-6" />
          </div>
          <div>
            <h1 className="font-bold text-base text-slate-100 leading-tight">Recicla ERP</h1>
            <span className="text-xs text-emerald-400 font-medium">Gestão Cooperativa</span>
          </div>
        </div>

        {/* Links de Navegação */}
        <nav className="flex-1 p-4 space-y-1">
          {menus.map((item) => {
            const Icone = item.icone;
            const ativo = telaAtiva === item.id;
            return (
              <button
                key={item.id}
                onClick={() => setTelaAtiva(item.id)}
                className={`w-full flex items-center gap-3 px-4 py-3 rounded-xl font-medium text-sm transition-all $
                  ativo
                    ? 'bg-emerald-600 text-white shadow-md'
                    : 'text-slate-400 hover:bg-slate-800 hover:text-slate-200'
                }`>
                <Icone className="w-5 h-5" />
                {item.rotulo}
              </button>
            );
          })}
        </nav>

        {/* Rodapé da Sidebar */}
        <div className="p-4 border-t border-slate-800 text-xs text-slate-500 text-center">
          v1.0.0 • Cooperativa Ativa
        </div>
      </aside>

      {/* Menu Mobile - Gaveta Superior */}
      <div className="md:hidden fixed top-0 left-0 right-0 bg-slate-900 text-white z-50 p-4 flex justify-between ite
        <div className="flex items-center gap-2">
          <Recycle className="w-6 h-6 text-emerald-400" />
          <span className="font-bold text-sm">Recicla ERP</span>
        </div>
        <button onClick={() => setMenuAberto(!menuAberto)} className="p-1">
          {menuAberto ? <X className="w-6 h-6" /> : <Menu className="w-6 h-6" />}
        </button>
      </div>

      {/* Drawer Overlay no Mobile */}
      {menuAberto && (
        <div className="md:hidden fixed inset-0 bg-slate-900/95 text-white z-40 pt-20 p-6 space-y-3">
          {menus.map((item) => {
            const Icone = item.icone;
            return (
              <button
                key={item.id}
                onClick={() => {
                  setTelaAtiva(item.id);

```

```

        setMenuAberto(false);
      }}
      className={`w-full flex items-center gap-4 p-4 rounded-xl font-semibold text-lg ${
        telaAtiva === item.id ? 'bg-emerald-600 text-white' : 'text-slate-300'
      }`}
    >
      <Icône className="w-6 h-6" />
      {item.rotulo}
    </button>
  );
  })}
</div>
)}

{/* Área de Conteúdo Principal */}
<main className="flex-1 overflow-y-auto pt-16 md:pt-0">
  {renderizarTela()}
</main>

</div>
);
}

```

User prompt: Montar os arquivos package.json do frontend e backend com as dependências necessárias.

Response: Configuração dos arquivos package.json para o projeto ERP, contendo os scripts de execução e bibliotecas necessárias para o Back-end e Front-end.

Back-end (backend/package.json)

```

{
  "name": "recicla-erp-backend",
  "version": "1.0.0",
  "description": "API Node.js para gestão financeira e rateio de cooperativas de reciclagem",
  "main": "src/server.js",
  "scripts": {
    "start": "node src/server.js",
    "dev": "nodemon src/server.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2",
    "pg": "^8.11.5"
  },
  "devDependencies": {
    "nodemon": "^3.1.0"
  }
}

```

Front-end (frontend/package.json)

```

{
  "name": "recicla-erp-frontend",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "lucide-react": "^0.378.0",
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  },
  "devDependencies": {
    "@types/react": "^18.3.1",
    "@types/react-dom": "^18.3.0",
    "@vitejs/plugin-react": "^4.2.1",
    "autoprefixer": "^10.4.19",
    "postcss": "^8.4.38",
    "tailwindcss": "^3.4.3",
    "vite": "^5.2.11"
  }
}

```

User prompt: Quais são os passos e serviços recomendados para hospedar este ERP gratuitamente na nuvem?

Response: Você pode hospedar o ERP da cooperativa com custo zero usando planos gratuitos de plataformas modernas de Nuvem (PaaS e DBaaS).

Mapeamento da Infraestrutura Gratuita

- **Banco de Dados (PostgreSQL): Render ou Supabase**
 - *O que oferecem:* Instância gerenciada do PostgreSQL com até 500 MB de armazenamento (suficiente para milhares de lançamentos da cooperativa).
- **Back-end (Node.js/Express): Render ou Koyeb**
 - *O que oferecem:* Hospedagem para aplicações Node.js integrada ao GitHub com deploy automático a cada commit.
- **Front-end (React/Vite): Vercel ou Netlify**
 - *O que oferecem:* Hospedagem rápida para aplicações React com SSL/HTTPS gratuito e distribuição via CDN global.

Passo a Passo para o Deploy

1. Subir o Código no GitHub Crie um repositório no GitHub (público ou privado) e faça a entrega das pastas backend/ e frontend/.

2. Criar o Banco de Dados no Render

1. Crie uma conta no render.com.
2. Vá em **New +** → **PostgreSQL**.
3. Copie a **Internal/External Database URL** gerada e execute o script `init.sql` no banco para criar as tabelas.

3. Hospedar a API Node.js no Render

1. No Render, clique em **New +** → **Web Service**.
2. Conecte o repositório do GitHub e selecione a pasta backend.
3. Defina os comandos:
 - **Build Command:** `npm install`
 - **Start Command:** `node src/server.js`
4. Na seção **Environment Variables**, adicione a variável `DATABASE_URL` com a string de conexão do PostgreSQL.
5. Copie a URL pública do seu Back-end (ex: `[https://recicla-api.onrender.com](https://recicla-api.onrender.com)`).

4. Hospedar o Front-end React na Vercel

1. Crie uma conta na vercel.com e importe o repositório.
 2. Indique que a pasta raiz do projeto é a `/frontend`.
 3. Em **Environment Variables**, adicione:
 - `VITE_API_URL = [https://recicla-api.onrender.com/api](https://recicla-api.onrender.com/api)`
 4. Clique em **Deploy**. A Vercel vai gerar a URL do sistema (ex: `[https://recicla-erp.vercel.app](https://recicla-erp.vercel.app)`).
-